LAB SHEET 7

Part A: DDL (Data Definition Language)

Task

A1 Create a table faculty with columns faculty_id (primary key), name (not null), email (unique), salary and dept_id (foreign key to departments).

A2 Add a new column phone (VARCHAR(15)) to faculty using ALTER TABLE.

A3 Rename the column phone to contact_no.

A4 Drop the column contact_no.

A5 Remove the faculty table completely, then explain the difference between DROP, TRUNCATE and DELETE.

Part B: DML (Data Manipulation Language)

Task

B1 Insert a new student: (109, 'Neha', 20, 'Roorkee', 1).

B2 Insert a student without specifying city and observe the value stored.

B3 Update the city of student 105 to 'Roorkee'.

B4 Increase marks by 5 for all enrollments in course 203.

B5 Delete the student with student_id = 109 (and any other row you added for testing).

B6 Try inserting a student with age 15. Record the error message and explain which constraint caused it.

Part C: Querying with SELECT

Task

C1 Display all columns of all students.

C2 Display only the name and city of all students.

C3 List students from Roorkee.

C4 List students aged between 20 and 22 (use BETWEEN).

C5 List students from Roorkee or Dehradun (use IN).

C6 List students whose name starts with 'A' (use LIKE).

C7 List students who have no department assigned (use IS NULL).

C8 Display distinct cities of all students.

C9 Display students sorted by age in descending order, then by name ascending.

C10 Display the top 3 oldest students (LIMIT). C11 Display name as Student Name and age as Age using aliases.

Part D: Functions and Aggregation

Task

D1 Find the total number of students.

D2 Find the average, minimum and maximum marks in the enrollments table.

D3 Count the number of students in each city.

D4 Find the average marks for each course.

D5 Show only those cities that have more than one student (HAVING).

D6 Find the total credits offered by each department.

D7 Display student names in uppercase and the length of each name (UPPER, LENGTH).

D8 Explain why WHERE cannot be used with aggregate functions, and what HAVING does differently.

Part E: Joins

Task

E1 Display each student's name along with their department name (INNER JOIN).

E2 Display all students, including those without a department (LEFT JOIN). Which student shows NULL?

E3 Display all departments, including those with no students (RIGHT JOIN or LEFT JOIN with tables swapped).

E4 Display student name, course title and marks by joining students, enrollments and courses.

E5 List the names of students enrolled in 'Data Structures'.

E6 Find students who are not enrolled in any course (LEFT JOIN with IS NULL).

E7 Display the number of students enrolled in each course (include courses with zero enrollments).

E8 Display each department with its number of students and its average age.

Part F: Subqueries

Task

F1 Find students whose age is greater than the average age.

F2 Find the student(s) with the highest marks in any course.

F3 List courses that have at least one enrollment (IN or EXISTS).

F4 List students enrolled in more than 2 courses.

F5 Find the department with the highest number of students.

F6 For each course, list students who scored above that course's average (correlated subquery).

Part G: Indexes, Views and Transactions

Task

G1 Create an index on students(city). Run EXPLAIN SELECT * FROM students WHERE city = 'Roorkee'; and note what changes.

G2 Create a view student_marks that shows student name, course title and marks.

G3 Query the view to list students scoring above 80.

G4 Drop the view and the index.

G5 Run a transaction: START TRANSACTION;, update marks for student 101 in course 201 to 100, then run ROLLBACK;. Check that the value is unchanged.

G6 Repeat G5 using COMMIT; instead and confirm the change is permanent

Part H: Challenge Problems (Optional)

1. Display a rank list of students by their total marks, with ties handled using RANK() or DENSE_RANK().

2. Assign a grade to each enrollment using CASE: 90 and above = A, 75 to 89 = B, 60 to 74 = C, below 60 = D.

3. Find pairs of students from the same city (self join).

4. Find the second highest marks in the enrollments table without using LIMIT. low the same patterns as the examples above.

Solutions -> Setup Database & Insert Data

```sql
CREATE DATABASE college_db;
USE college_db;
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 17),
    city VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    title VARCHAR(80) NOT NULL,
    credits INT DEFAULT 3,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

```sql
INSERT INTO departments VALUES (1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');
INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);
INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);
INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),
(103,203,81),
(104,201,95),(104,202,89),
(105,204,58),
(106,203,74),
(108,203,66);
```

Part A: DDL (Data Definition Language)

A1:

```sql
CREATE TABLE faculty (
    faculty_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10, 2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

A2:

```sql
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
```

A3

```sql
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
```

A4:

```sql
ALTER TABLE faculty DROP COLUMN contact_no;
```

A5:

```sql
DROP TABLE faculty;
```

Part B: DML (Data Manipulation Language)

B1:

```sql
INSERT INTO students (student_id, name, age, city, dept_id)
VALUES (109, 'Neha', 20, 'Roorkee', 1);
```

B2:

```sql
INSERT INTO students (student_id, name, age, dept_id)
VALUES (110, 'Amit', 22, 2);
```

B3:

```sql
UPDATE students
SET city = 'Roorkee'
WHERE student_id = 105;
```

B4:

```sql
UPDATE enrollments
SET marks = marks + 5
WHERE course_id = 203;
```

B5:

```sql
DELETE FROM students
WHERE student_id IN (109, 110);
```

B6:

```
66 •  INSERT INTO students (student_id, name, age, city, dept_id)
67    VALUES (111, 'Anuj', 15, 'Delhi', 1);
```

Error Message:

```
21  14:18:19  INSERT INTO students (student_id, name, age, city, dept_id) VALUES (111, 'Anuj', 15, 'Delhi', 1)   Error Code: 3819. Check constraint 'students_chk_1' is violated.   0.031 sec
```

Part C: Querying with SELECT

C1:

```
80 •  SELECT * FROM students;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	NULL
108	Isha	19	Roorkee	2
NULL	NULL	NULL	NULL	NULL

C2:

```
81 •  SELECT name, city FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	city
Aarav	Roorkee
Diya	Dehradun
Kabir	Haridwar
Meera	Roorkee
Rohan	Roorkee
Sana	Dehradun
Vikram	Jaipur
Isha	Roorkee

C3:

```
82 •  SELECT * FROM students
83    WHERE city = 'Roorkee';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2
NULL	NULL	NULL	NULL	NULL

C4:

```
84 •  SELECT * FROM students
85    WHERE age BETWEEN 20 AND 22;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
NULL	NULL	NULL	NULL	NULL

C5:

```
86 •  SELECT * FROM students
87    WHERE city IN ('Roorkee', 'Dehradun');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
108	Isha	19	Roorkee	2
NULL	NULL	NULL	NULL	NULL

C6:

```
88 •  SELECT * FROM students
89    WHERE name LIKE 'A%';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
NULL	NULL	NULL	NULL	NULL

C7:

```
90 •  SELECT * FROM students
91    WHERE dept_id IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL
NULL	NULL	NULL	NULL	NULL

C8:

```
92 •  SELECT DISTINCT city FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city
Roorkee
Dehradun
Haridwar
Jaipur

C9:

```
93 •  SELECT * FROM students
94    ORDER BY age DESC, name ASC;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1
106	Sana	21	Dehradun	2
101	Aarav	20	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2
103	Kabir	19	Haridwar	2
NULL	NULL	NULL	NULL	NULL

C10:

```
95 •  SELECT * FROM students
96    ORDER BY age DESC
97    LIMIT 3;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	NULL
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1
NULL	NULL	NULL	NULL	NULL

C11:

```
98 •  SELECT name AS "Student Name", age AS "Age"
99    FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Student Name	Age
Aarav	20
Diya	21
Kabir	19
Meera	22
Rohan	20
Sana	21
Vikram	23
Isha	19

Part D: Functions and Aggregation

D1:

```
100 •  SELECT COUNT(*) AS total_students FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_students
8

D2:

```
101 •  SELECT
102        AVG(marks) AS avg_marks,
103        MIN(marks) AS min_marks,
104        MAX(marks) AS max_marks
105    FROM enrollments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_marks	min_marks	max_marks
79.2727	58	95

D3:

```
106 •  SELECT city, COUNT(*) AS student_count
107    FROM students
108    GROUP BY city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	student_count
Roorkee	4
Dehradun	2
Haridwar	1
Jaipur	1

D4:

```
109 •  SELECT course_id, AVG(marks) AS avg_marks
110    FROM enrollments
111    GROUP BY course_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

course_id	avg_marks
201	83.3333
202	82.5000
203	78.6667
204	58.0000
205	81.5000

D5:

```
112 •  SELECT city, COUNT(*) AS student_count
113    FROM students
114    GROUP BY city
115    HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	student_count
Roorkee	4
Dehradun	2

D6:

```
116 •  SELECT dept_id, SUM(credits) AS total_credits
117    FROM courses
118    GROUP BY dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_id	total_credits
1	12
2	3
3	3

D7:

```
119 •  SELECT UPPER(name) AS upper_name, LENGTH(name) AS name_length
120    FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

upper_name	name_length
AARAV	5
DIYA	4
KABIR	5
MEERA	5
ROHAN	5
SANA	4
VIKRAM	6
ISHA	4

Part E: Joins

E1:

```
121 •   SELECT s.name AS student_name, d.dept_name
122     FROM students s
123     INNER JOIN departments d ON s.dept_id = d.dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_name	dept_name
Aarav	CSE
Diya	CSE
Meera	CSE
Kabir	ECE
Sana	ECE
Isha	ECE
Rohan	Mechanical

E2:

```
124 •   SELECT s.name AS student_name, d.dept_name
125     FROM students s
126     LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Vikram	NULL
Isha	ECE

E3:

```
127 •   SELECT d.dept_name, s.name AS student_name
128     FROM departments d
129     LEFT JOIN students s ON d.dept_id = s.dept_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	student_name
Civil	NULL
CSE	Aarav
CSE	Diya
CSE	Meera
ECE	Kabir
ECE	Sana
ECE	Isha
Mechanical	Rohan

E4:

```
130 •  SELECT s.name AS student_name, c.title AS course_title, e.marks
131    FROM students s
132    JOIN enrollments e ON s.student_id = e.student_id
133    JOIN courses c ON e.course_id = c.course_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_name	course_title	marks
Aarav	Data Structures	88
Diya	Data Structures	67
Meera	Data Structures	95
Aarav	Operating Systems	76
Meera	Operating Systems	89
Kabir	Digital Electronics	86
Sana	Digital Electronics	79
Isha	Digital Electronics	71
Rohan	Thermodynamics	58

E5:

```
134 •  SELECT s.name
135    FROM students s
136    JOIN enrollments e ON s.student_id = e.student_id
137    JOIN courses c ON e.course_id = c.course_id
138    WHERE c.title = 'Data Structures';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Aarav
Diya
Meera

E6:

```
139 •  SELECT s.name
140    FROM students s
141    LEFT JOIN enrollments e ON s.student_id = e.student_id
142    WHERE e.course_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Vikram

E7:

```
143 •  SELECT c.course_id, c.title, COUNT(e.student_id) AS enrolled_students
144    FROM courses c
145    LEFT JOIN enrollments e ON c.course_id = e.course_id
146    GROUP BY c.course_id, c.title;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

course_id	title	enrolled_students
201	Data Structures	3
202	Operating Systems	2
203	Digital Electronics	3
204	Thermodynamics	1
205	Machine Learning	2

E8:

```
147 •  SELECT d.dept_id, d.dept_name, COUNT(s.student_id) AS total_students, AVG(s.age) AS avg_age
148    FROM departments d
149    LEFT JOIN students s ON d.dept_id = s.dept_id
150    GROUP BY d.dept_id, d.dept_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_id	dept_name	total_students	avg_age
4	Civil	0	NULL
1	CSE	3	21.0000
2	ECE	3	19.6667
3	Mechanical	1	20.0000

Part F: Subqueries

F1:

```
151 •  SELECT * FROM students
152    WHERE age > (SELECT AVG(age) FROM students);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	name	age	city	dept_id
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	NULL
NULL	NULL	NULL	NULL	NULL

F2:

```
153 •  SELECT s.*, e.marks
154    FROM students s
155    JOIN enrollments e ON s.student_id = e.student_id
156    WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_id	name	age	city	dept_id	marks
104	Meera	22	Roorkee	1	95

F3:

```
157 •  SELECT * FROM courses
158    WHERE course_id IN (SELECT DISTINCT course_id FROM enrollments);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

course_id	title	credits	dept_id
201	Data Structures	4	1
202	Operating Systems	4	1
203	Digital Electronics	3	2
204	Thermodynamics	3	3
205	Machine Learning	4	1
NULL	NULL	NULL	NULL

F4:

```
159 •  SELECT student_id, COUNT(course_id) AS total_courses
160    FROM enrollments
161    GROUP BY student_id
162    HAVING COUNT(course_id) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_id	total_courses
101	3

F5:

```
163 •  SELECT dept_id, COUNT(*) AS student_count
164    FROM students
165    WHERE dept_id IS NOT NULL
166    GROUP BY dept_id
167    ORDER BY student_count DESC
168    LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

dept_id	student_count
1	3

F6:

```sql
SELECT e1.course_id, s.name, e1.marks
FROM enrollments e1
JOIN students s ON e1.student_id = s.student_id
WHERE e1.marks > (
    SELECT AVG(e2.marks)
    FROM enrollments e2
    WHERE e2.course_id = e1.course_id
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

course_id	name	marks
201	Aarav	88
205	Aarav	91
203	Kabir	86
201	Meera	95
202	Meera	89
203	Sana	79

Part G: Indexes, Views and Transactions

G1:

```sql
CREATE INDEX idx_students_city ON students(city);
EXPLAIN SELECT * FROM students WHERE city = 'Roorkee';
```

G2:

```sql
CREATE VIEW student_marks AS
SELECT s.name AS student_name, c.title AS course_title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

G3:

```sql
SELECT * FROM student_marks
WHERE marks > 80;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_name	course_title	marks
Aarav	Data Structures	88
Aarav	Machine Learning	91
Kabir	Digital Electronics	86
Meera	Data Structures	95
Meera	Operating Systems	89

G4:

```sql
DROP VIEW student_marks;
DROP INDEX idx_students_city ON students;
```

G5:

```sql
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
ROLLBACK;
SELECT marks FROM enrollments WHERE student_id = 101 AND course_id = 201;
```

G6:

```sql
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
COMMIT;
SELECT marks FROM enrollments WHERE student_id = 101 AND course_id = 201;
```